

# Delay-prioritized Task Scheduling with Load Balancing in Computing Power Networks

Li Feng<sup>\*</sup>, Renchao Xie<sup>\*†</sup>, Qinqin Tang<sup>\*</sup>, Tao Huang<sup>\*†</sup>

<sup>\*</sup>State Key Laboratory of Networking and Switching Technology,  
Beijing University of Posts and Telecommunications, Beijing, China

<sup>†</sup>Purple Mountain Laboratories, Nanjing, China

Email: {fengli, renchao\_xie, qqtang, htao}@bupt.edu.cn

**Abstract**—In the era of data-driven intelligent Internet, efficient utilization of computing power is paramount. Yet, current cloud-edge collaboration architectures face challenges with computing power isolation, adversely affecting efficiency and user experience. Computing Power Networks (CPNs) leverage networks with cloud-native applications to connect and manage resources, offering a blueprint for a collaborative computing ecosystem. In the CPN, scheduling stands as a pivotal function. However, conventional scheduling often neglects the interplay between computing and networks. To rectify this, we present a collaborative task scheduling system in CPNs that simultaneously contemplates the selections of computing nodes and network links. Aiming to maintain a balanced load for both computing and network resources, we formulate the scheduling challenge as a Constrained Markov Decision Process (CMDP). This approach focuses on optimizing both execution delay and success rate of computing tasks with load balancing constraints in CPNs. To facilitate the resolution of the CMDP, we introduce a Lyapunov-optimized Deep Reinforcement Learning (DRL) algorithm, which reconfigures the long-term constraint into immediate optimization. We provide numerical results to demonstrate the effectiveness of our suggested policy and algorithm.

**Index Terms**—Computing Power Networks, collaborative task scheduling, delay optimization, load balancing

## I. INTRODUCTION

The rapid advancement of information technology has catalyzed the emergence of cutting-edge intelligent applications such as Digital Twins (DT) and Artificial Intelligence Generated Content (AIGC), driven by the growth in data, algorithms, and computing power. According to IDC's projections, global data volumes are set to grow at a Compound Annual Growth Rate (CAGR) of 23%, reaching 181ZB by 2025 [1]. Moreover, the global AI computing market is projected to expand from \$19.5 billion in 2022 to \$34.6 billion by 2026 [2]. Amid this swift socio-economic evolution, computing power emerges as a pivotal catalyst.

With the progression of computing power and increasing demands for low delay and enhanced security, there has been a shift from centralized cloud-based computing to decentralized edge computing tailored to specific service needs [3]. While this transition leads to a three-tiered cloud-edge-end architecture, cloud, edge, and endpoints often remain isolated, resulting in suboptimal resource utilization. Fortunately, Computing Power Networks (CPNs) mitigate these challenges by

utilizing networks to connect and coordinate these widespread and heterogeneous resources, promoting a cohesive system and abstracting resource disparities for users [4]–[7]. Recognizing these remarkable benefits, CPNs have garnered significant attention from both industry and academia. In September 2021, ITU-T introduced its inaugural CPNs recommendation, Y.2501 [5], detailing the framework and architecture of CPNs. Tang *et al.* in [6] integrated Intent Networks to propose a Service Intent-aware Task Scheduling (SIaTS) framework for CPNs. Similarly, Yang *et al.* in [7] created an Intercloud Broker for Sky Computing that allows users to view the cloud ecosystem as a more integrated sky of computing. With the comprehensive integration of computing and network resources in CPNs, challenges like computing power isolation are addressed, paving the way for a more collaborative and responsive computational ecosystem.

In the realm of CPNs, the orchestrating and scheduling functionality, often termed the “computational brain” [4], has attracted significant attention from researchers. However, traditional cloud-edge scheduling primarily focuses on the selection of heterogeneous nodes and optimization of multi-objective combinations, often sidelining network considerations [8]–[10]. The CPN profoundly integrates computing and networks, making scheduling not just about addressing heterogeneous computing nodes but also managing intricate network links. This necessitates distributing computing tasks across network links to avert cascading issues, such as increased delay and packet loss due to network congestion. Furthermore, given the high concurrency and uneven distribution of computing tasks in the CPN, prioritizing load balancing between computing and network resources is pivotal for the sustained stability of the system, all while enhancing resource utilization.

To address the challenges above, this paper investigates the scheduling of computing tasks in the CPN where computing and networks are deeply integrated. Aiming to optimize the utilization of both computing and network resources, we make simultaneous decisions on the allocation of computing tasks across multiple computing nodes and network links while scheduling. Additionally, we establish load balancing models for both entities to assess the system's long-term reliability. To ensure the timely and reliable execution of computing tasks and minimize average delay, we introduce an effective

delay-prioritized policy to boost the task success rate. Our contributions are as follows.

- We develop a delay-prioritized task scheduling system with load balancing in the CPN. We then formulate the scheduling issue as a Constrained Markov Decision Process (CMDP) to reduce delay and packet loss while ensuring long-term load balancing.
- To address the CMDP challenge, we use Deep Reinforcement Learning (DRL) and, with Lyapunov stability theory, convert the long-term load balancing constraint into immediate Lyapunov drift optimization.
- Through comprehensive simulations, we compare our algorithm with four benchmarks, underscoring the superiority of our policy and approach.

The rest of the article is structured as follows. Section II outlines the system model. Section III details the CMDP problem and our proposed algorithm. Simulation results are presented in Section IV, and Section V offers the conclusion.

## II. SYSTEM MODEL

In this section, we design a collaborative scheduling system with a dynamic delay-prioritized policy and present the execution delay and load balancing models for both computing and network resources.

### A. Scheduling system

**System Framework:** Fig. 1 illustrates a collaborative scheduling system in CPNs. Computing tasks access the CPN through ubiquitous networks at the Access Layer. The CPN Layer contains various heterogeneous devices, each with unique computing and cache resources. For user simplicity, we refer to them as CPN nodes, connected via CPN routers. The Scheduling Layer is composed of three key components, namely the Resource Catalog, Scheduling Optimizer, and Scheduling Executor. Given the continuous updates in CPNs, we assume that the system operates in discrete time. We divide the continuous scheduling time into fixed-duration slots  $\mathcal{T} = \{1, 2, \dots, T\}$ . Let  $\Delta t$  denote the duration of each slot, and  $t \in \mathcal{T}$  denote the index.

**Network Links:** Consider a CPN  $\mathbb{N}(\mathcal{N}, \mathcal{R}, \mathcal{L})$  with  $\mathcal{N} = [N] \triangleq \{1, 2, \dots, N\}$  representing the set of CPN nodes,  $\mathcal{R} = [R] \triangleq \{1, 2, \dots, R\}$  representing the set of CPN routers, and  $\mathcal{L} = [L] \triangleq \{1, 2, \dots, L\}$  representing the set of network links. For link  $l = (r, r') \in \mathcal{L}$ , let  $u_l$  be the maximum network link capacity, and  $v_l(t)$  be the real-time network transmission rate, where  $r \in \mathcal{R}$  and  $r' \in \mathcal{R}$  are the connected CPN routers. The network employs the Segment Routing (SR) protocol. When multiple SR paths exist between the source and destination, the forwarding path is determined by referencing the SR-Traffic Engineering (SR-TE) policy [11], effectively reducing the forwarding pressure on network devices. Denote  $\mathcal{P}_{n,n'} = \{\mathcal{P}_{n,n'}^1, \mathcal{P}_{n,n'}^2, \dots, \mathcal{P}_{n,n'}^H\}$  the candidate SR paths between the source CPN node  $n \in \mathcal{N}$  and the destination CPN node  $n' \in \mathcal{N}$ , while  $\mathcal{P}_{n,n'}^h \in \mathcal{P}_{n,n'} = \{n, l, \dots, n'\}$  representing the  $h$ -th SR path.

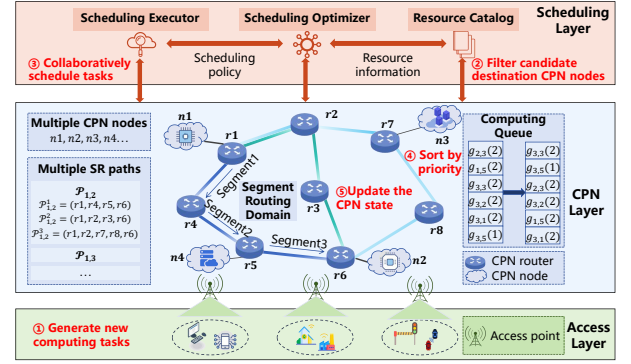


Fig. 1: A collaborative scheduling system in CPNs.

**Computing Tasks:** At every time slot  $t$ ,  $\mathcal{G}(t) = \{\mathcal{G}_1(t), \mathcal{G}_2(t), \dots, \mathcal{G}_N(t)\}$  denotes the set of newly accessed tasks, while  $\mathcal{G}_n(t) \in \mathcal{G}(t)$  being a sequence of i.i.d. bounded random variables with  $\mathbb{E}[\mathcal{G}_n(t)] = \lambda$ , and  $|\mathcal{G}_n(t)| \leq K$ . Let  $\mathcal{G}_n(t) = \{g_{n,1}(t), g_{n,2}(t), \dots, g_{n,K}(t)\}$  be the new tasks on CPN node  $n$ , where  $g_{n,k}(t) \in \mathcal{G}_n(t)$  represents the  $k$ -th newly arrived task. Denote the task tuple as  $g_{n,k}(t) = (s_{n,k}(t), d_{n,k}(t), o_{n,k}(t), w_{n,k}(t))$ , where  $s_{n,k}(t)$  represents the resource type,  $d_{n,k}(t)$  is the data volume,  $o_{n,k}(t)$  is the computing volume, and  $w_{n,k}(t)$  represents the tolerable delay.

**CPN Nodes:** Let  $n = (f_n, m_n, c_n, s_n)$  be the properties of CPN node  $n \in \mathcal{N}$ , where  $f_n, m_n, c_n, s_n$  represents the maximum computing power, the maximum cache resources, the connected CPN router, and the type of computing resource, respectively. Each CPN node  $n$  maintains both a buffer queue  $\mathcal{B}_n(t)$  and a computing queue  $\mathcal{C}_n(t)$  at time slot  $t$ . For new tasks scheduled on  $n$ , they enter the buffer queue and record the data volume as  $\mathcal{B}_n^{\text{new}}(t)$ . Tasks with a total data volume of  $\mathcal{B}_n^{\text{out}}(t)$  are taken out from the buffer queue to the computing queue, with the computing volume of  $\mathcal{C}_n^{\text{in}}(t)$ . Before a new time slot, the computing volume of completed tasks is counted as  $\mathcal{C}_n^{\text{fin}}(t)$ . Due to limited resources, let  $\mathcal{B}_n^{\text{ove}}(t)$  represent the data volume overflowed on the buffer queue, and  $\mathcal{C}_n^{\text{dis}}(t)$  represent the computing volume for tasks completed overtime. Across two consecutive time slots, they are updated as follows

$$\mathcal{B}_n(t+1) = \min \{ \mathcal{B}_n(t) + \mathcal{B}_n^{\text{new}}(t) - \mathcal{B}_n^{\text{out}}(t), m_n \}, \quad (1)$$

$$\mathcal{C}_n(t+1) = \mathcal{C}_n(t) + \mathcal{C}_n^{\text{in}}(t) - \mathcal{C}_n^{\text{fin}}(t). \quad (2)$$

**Scheduling Process:** The scheduling process for newly accessed tasks at time slot  $t$  can be summarized as follows.

- **Information Filtering:** For each task, the system filters mismatched or unreachable CPN nodes from the Resource Catalog based on task attributes.
- **CPNs Dynamic Scheduling:** We address the dynamic multi-sources, multi-destinations, and multi-paths scenarios. The Scheduling Optimizer makes the scheduling policy for new tasks, considering both selections of CPN nodes and SR paths. Denote the scheduling policy of computing task  $g_{n,k}(t)$  as  $a_{n,k}(t) = (a_{n,k}^{\text{node}}(t), a_{n,k}^{\text{path}}(t))$ ,

which indicates that  $g_{n,k}(t)$  is transmitted along SR path  $a_{n,k}^{\text{path}}(t) \in \mathcal{P}_{n,n'}$  to CPN node  $n' = a_{n,k}^{\text{node}}(t)$  for execution.

- **Tasks Priority Sorting:** Considering resource limitations and time-sensitive needs, we employ a delay-prioritized policy. Unlike traditional methods focusing solely on average delay, we prioritize tasks with lower tolerable delay or longer queuing delay as follows

$$PR_{n,k}(t) = (t^* - t) - w_{n,k}(t), \quad (3)$$

where  $t^*$  indicates the executing time slot of  $g_{n,k}(t)$ .

- **CPNs States Updating:** Before a new scheduling slot, updating the states of network links, CPN nodes, and computing tasks is crucial.

### B. Execution Delay Model

**Computing Delay:** Delay is essential for gauging Quality of Service (QoS). When locally executing task  $g_{n,k}(t)$  on CPN node  $n$ , the delay comprises both computing and queue waiting delay, i.e.,

$$D_{n,k}^{\text{loc}}(t) = \frac{o_{n,k}(t)}{f_n} + D_{n,k}^{\text{wait}}(t). \quad (4)$$

The first term denotes the computing delay, and  $D_{n,k}^{\text{wait}}(t)$  represents the queue waiting delay. With tasks prioritized in the queue, the average queue waiting delay is expressed as

$$D_{n,k}^{\text{wait}}(t) = \frac{|C_n(t^*)|}{2f_n}. \quad (5)$$

**Transmission Delay:** While  $g_{n,k}(t)$  routing via  $\mathcal{P}_{n,n'}^h$  to  $n'$ , the transmission delay is given by

$$D_{n,k}^{\text{tran}}(t) = d_{n,k}(t) \cdot \sum_{l \in \mathcal{P}_{n,n'}^h} \frac{1}{v_l(t)}. \quad (6)$$

Hence, for tasks not executed on the accessed CPN node, the execution delay can be calculated as

$$D_{n,k}^{\text{off}}(t) = \frac{o_{n,k}(t)}{f_{n'}} + \frac{|C_{n'}(t^*)|}{2f_{n'}} + D_{n,k}^{\text{tran}}(t). \quad (7)$$

In summary, the total delay for task  $g_{n,k}(t)$  executed on the CPN node  $n'$  can be formulated as

$$D_{n,k}(t) = D_{n,k}^{\text{loc}}(t) \cdot \{n' = n\} + D_{n,k}^{\text{off}}(t) \cdot \{n' \neq n\}. \quad (8)$$

### C. Load Balancing Model

Generally, tasks favor CPN nodes with higher computing power via the shortest SR path, potentially increasing delay or packet loss. Thus, maintaining long-term load balancing is crucial for enhancing resource utilization and ensuring system stability.

**Balance of CPN Nodes:** We measure the load of CPN node  $n$  as a weighted combination of utilized computing and cache resources as follows

$$L_n(t) = \iota_1 L_n^{\text{com}}(t) + \iota_2 L_n^{\text{cac}}(t), \text{ s.t. } \iota_1 + \iota_2 = 1, \quad (9)$$

where  $\iota_1, \iota_2$  are trade-off factors.  $L_n^{\text{com}}(t)$  and  $L_n^{\text{cac}}(t)$  represents the load of computing resources and cache resources, respectively. They can be calculated as

$$L_n^{\text{com}}(t) = \min \left\{ \frac{|C_n(t)|}{\Delta t \cdot f_n}, 1 \right\}, \quad (10)$$

$$L_n^{\text{cac}}(t) = \min \left\{ \frac{|\mathcal{B}_n(t)|}{m_n}, 1 \right\}. \quad (11)$$

Typically, a smaller value indicates a lighter load. When computing or cache resources are exhausted, the value becomes 1, a design made for upcoming problem transformations.

We then utilize the standard deviation to assess the load balancing performance of all CPN nodes as follows

$$\mathcal{L}^{\text{node}}(t) = \sqrt{\frac{\sum_{n \in \mathcal{N}} \left( L_n(t) - \overline{L_n(t)} \right)^2}{N}}, \quad (12)$$

where  $\overline{L_n(t)}$  denotes the average load of all CPN nodes.

**Balance of Network Links:** We gauge the load of network link  $l$  using its utilization as

$$L_l(t) = \frac{\mathcal{U}_l(t)}{u_l}. \quad (13)$$

$\mathcal{U}_l(t)$  is the real-time data volume on the link that evolves as follows

$$\mathcal{U}_l(t+1) = \min \{ \mathcal{U}_l(t) + \mathcal{U}_l^{\text{new}}(t) - \mathcal{U}_l^{\text{suc}}(t), u_l \}, \quad (14)$$

where  $\mathcal{U}_l^{\text{new}}(t)$  and  $\mathcal{U}_l^{\text{suc}}(t)$  represent the data volume of newly uploaded tasks and successfully transmitted tasks, respectively.

Each  $L_l(t)$  ranges between (0, 1), and is set to 1 when the link is overloaded, aligning with future problem transformations. Then the load balancing performance of overall network links can be expressed as

$$\mathcal{L}^{\text{link}}(t) = \sqrt{\frac{\sum_{l \in \mathcal{L}} \left( L_l(t) - \overline{L_l(t)} \right)^2}{L}}, \quad (15)$$

where  $\overline{L_l(t)}$  represents the average load of all network links.

Hence, for the whole system, the load balancing performance is a weighted blend of  $\mathcal{L}^{\text{node}}$  and  $\mathcal{L}^{\text{link}}$ , expressed as

$$\mathcal{L}(t) = \iota_3 \cdot \mathcal{L}^{\text{node}}(t) + \iota_4 \cdot \mathcal{L}^{\text{link}}(t), \quad (16)$$

where  $\iota_3$  and  $\iota_4$  adjust the trade-off of  $\mathcal{L}^{\text{node}}(t)$  and  $\mathcal{L}^{\text{link}}(t)$ .

## III. A LYAPUNOV-OPTIMIZED DEEP REINFORCEMENT LEARNING ALGORITHM

In this section, we formulate the scheduling process as a CMDP problem, then use the Lyapunov stability theory to convert long-term constraints into immediate optimization objectives, and introduce a Lyapunov-optimized DRL algorithm.

### A. CMDP Formulation

Consistent load balancing is essential for system stability and efficiency. Treating it as an instant reward may cause system failure, whereas framing it as a long-term constraint ensures sustained resource optimization and stability. Thus, we formulate the scheduling process as a CMDP, with details provided below.

**States:** The state at time slot  $t$  is a composite of information from CPN nodes, network links, and computing tasks, i.e.,  $\mathbb{S}(t) = \{ \{ \mathcal{B}_n(t) \mid n \in \mathcal{N} \}, \{ C_n(t) \mid n \in \mathcal{N} \}, \{ \mathcal{U}_l(t) \mid l \in \mathcal{L} \}, \mathcal{G}(t) \}$ . (17)

**Actions:** The scheduling actions for all tasks is given by

$$\mathbb{A}(t) = \{ a_{n,k}(t) \mid n \in \mathcal{N}, k < K \}. \quad (18)$$

**Rewards:** With the system goal of minimizing execution delay and packet loss, we use the negative value of execution delay as an immediate reward and the success rate of tasks as an additional reward.

We can express the success rate of tasks executed on CPN node  $n$  as

$$R_n^{\text{suc}}(t) = 0.5 \cdot \left(1 - \frac{\mathcal{B}_n^{\text{ove}}(t)}{\mathcal{B}_n^{\text{new}}(t)}\right) + 0.5 \cdot \left(1 - \frac{\mathcal{C}_n^{\text{dis}}(t)}{\mathcal{C}_n^{\text{fin}}(t)}\right), \quad (19)$$

where the first term indicates the success rate of entering the buffer queue, and the second term represents the successful computing rate on the computing queue. Thus, the reward can be expressed as

$$\mathbb{R}(t) = \gamma_1 \sum_{n=1}^N R_n^{\text{suc}}(t) - \gamma_2 \sum_{n=1}^N \sum_{k=1}^K D_{n,k}(t), \quad (20)$$

where  $\gamma_1$  and  $\gamma_2$  are positive parameters to adjust the trade-off.

**States Updating:** In the scheduling process, different actions result in varied state changes of network links, CPN nodes, and computing tasks, which can be represented as follows

$$\mathcal{B}_n^{\text{new}}(t) = \sum_{n'=1}^N \sum_{k=1}^K d_{n',k}(t) \cdot \{a_{n',k}^{\text{node}}(t) = n\}, \quad (21)$$

$$\mathcal{B}_n^{\text{ove}}(t) = \sum_{n'=1}^N \sum_{k=1}^K d_{n',k}(t) \cdot \{\mathcal{B}_n(t) + o_{n',k}(t) > m_n\}, \quad (22)$$

$$\mathcal{C}_n^{\text{dis}}(t) = \sum_{n'=1}^N \sum_{k=1}^K o_{n',k}(t) \cdot \{D_{n',k}(t) > w_{n',k}(t)\}, \quad (23)$$

$$\mathcal{U}_l^{\text{new}}(t) = \sum_{n=1}^N \sum_{k=1}^K d_{n,k}^t \cdot \{a_{n,k}^{\text{path}}(t) \ni l\}. \quad (24)$$

**Problem:** To realize the delay-prioritized task scheduling with load balancing in CPNs, this paper seeks a stationary policy  $\pi$  that jointly considers the selections of CPN nodes and SR paths to increase rewards. Thus, we formulate the CMDP problem as follows

$$\text{P1: } \max_{\pi} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T E_{\pi} [\mathbb{R}(t)], \quad (25)$$

$$\text{s.t. } \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathcal{L}(t) \leq \mathcal{L}^{\text{up}}, \quad (26)$$

where  $\mathcal{L}^{\text{up}}$  denotes the upper limit of long-term load balancing.

## B. Problem Transformation

DRL is a highly suitable algorithm for addressing MDP problems [12]. Nevertheless, the complexity introduced by constraints in CMDP problems presents significant challenges in utilizing DRL to identify optimal solutions within constrained temporal and spatial parameters.

Lyapunov stability [13], a concept from dynamics and control, elucidates long-term system stability. For a nonlinear system  $\dot{x} = f(x)$ , where  $f$  represents the system and  $x$  the state vector, the goal is to identify a stable equilibrium point  $x^*$  such that  $f(x^*) = 0$ .

For a globally asymptotically stable  $x^*$ , the Lyapunov function assesses the stability and exhibits both positive definiteness and negative definiteness of its time derivative, i.e.,

$$\begin{cases} L(x^*) = 0, \\ L(x) > 0, \forall x \neq x^*, \\ L(\dot{x}, t) < 0. \end{cases} \quad (27)$$

Hence, to address the CMDP problem and streamline DRL's solution, we employ Lyapunov stability theory to create a load balancing differential function, i.e.,

$$\begin{cases} F^t = 0, t = 1, \\ F^{t+1} = [\mathcal{L}(t+1) - \mathcal{L}^{\text{up}} + F^t]^+, t > 1. \end{cases} \quad (28)$$

The function accumulates the load balancing difference up to time slot  $t$ . Ideally, we would expect this function to converge to zero over time, indicating the system's equilibrium state.

Then, to establish the asymptotic stability of the system's load balancing, we construct a Lyapunov function, that is,

$$L(F^t) = \frac{1}{2} (F^t)^2. \quad (29)$$

This function is positive definite and effectively portrays the system's adherence to long-term load balancing constraints.

Finally, to drive the Lyapunov function towards zero over time, we introduce an instantaneous Lyapunov drift [14], which tracks changes between two consecutive time slots, i.e.,

$$\Delta(F^t) = L(F^{t+1}) - L(F^t). \quad (30)$$

The following derivation provides an upper bound for the Lyapunov drift

$$\begin{aligned} \Delta(F^t) &= L(F^{t+1}) - L(F^t) \\ &= \frac{1}{2} \left( (F^{t+1})^2 - (F^t)^2 \right) \\ &\leq \frac{1}{2} \left( (\mathcal{L}(t+1) - \mathcal{L}^{\text{up}} + F^t)^2 - (F^t)^2 \right) \\ &= \frac{1}{2} (\mathcal{L}(t+1) - \mathcal{L}^{\text{up}})^2 + F^t (\mathcal{L}(t+1) - \mathcal{L}^{\text{up}}) \\ &\leq C_{\text{max}} + F^t (\mathcal{L}(t+1) - \mathcal{L}^{\text{up}}). \end{aligned} \quad (31)$$

Since  $\mathcal{L}^{\text{up}}$  ranges between 0 and 1,  $C_{\text{max}}$  is a constant that can be expressed as  $C_{\text{max}} = \max \left\{ \frac{1}{2} (1 - \mathcal{L}^{\text{up}})^2, \frac{1}{2} (\mathcal{L}^{\text{up}})^2 \right\}$ .

Applying the Lyapunov stability optimization theory, we can transform the CMDP's long-term load balancing constraint into an optimization objective involving Lyapunov drift at each time slot. With this transformation, we incorporate the difference accumulation term  $F^t$  into the state, and integrate the drift as a penalty term  $-\Delta(F^t)$  within the reward function. This leads to a revised ordinary MDP, and problem P1 is consequently modified as follows

$$\begin{aligned} \text{P2: } \max_{\pi} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T & \left( \gamma_1 \sum_{n=1}^N E_{\pi} [R_n^{\text{suc}}(t)] - \right. \\ & \left. \gamma_2 \sum_{n=1}^N \sum_{k=1}^K E_{\pi} [D_{n,k}(t)] - \gamma_3 \cdot E_{\pi} [\Delta(F^t)] \right), \end{aligned} \quad (32)$$

where  $\gamma_3$  is also a positive parameter to fine-tune the trade-off.

### C. Lyapunov-optimized DRL Algorithm

As illustrated in Figure 2, we propose a Lyapunov-optimized Twin Delayed Deep Deterministic Policy Gradient (LTD3) algorithm. It mitigates the high estimation bias inherent in the Deep Deterministic Policy Gradient (DDPG) algorithm by utilizing twin critic networks, delayed policy updates and target policy smoothing [15], which contributes to formulating a more effective strategy for long-term scheduling.

The proposed algorithm comprises several episodes, with each episode containing multiple steps corresponding to scheduling time slots. It observes the state in the CPN and yields the scheduling policy. Below are the detailed steps.

**Initialization:** Randomly initialize the actor network  $\pi_\theta$ , two critic networks  $Q_{\theta_1}$ ,  $Q_{\theta_2}$ , and their corresponding target networks  $\pi_{\theta'}$ ,  $Q_{\theta'_1}$ , and  $Q_{\theta'_2}$ . Initialize the replay buffer, reset the environment, and obtain the initial state.

**Exploration and Storage:** At each slot, select an action  $a$  with exploration noise  $\epsilon$  to execute in the environment  $s$ , i.e.,

$$a \sim \pi_\phi(s) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma). \quad (33)$$

Observe the next state  $s'$  and the reward  $r$ , then store the tuple  $(s, a, r, s')$  to the replay buffer.

**Network Updating:** Upon meeting the update conditions, sample a mini-batch  $B$  from the replay buffer and choose an action  $\tilde{a}$  with policy noise  $\epsilon'$ , i.e.,

$$\tilde{a} = \pi_{\phi'}(s') + \epsilon', \epsilon' \sim \text{clip}(\mathcal{N}(0, \tilde{\omega}), -c, c). \quad (34)$$

Compute target values with discount factor  $\gamma$  by

$$y = r + \gamma \min_{i=1,2} \{Q_{\theta'_i}(s', \tilde{a})\}, \quad (35)$$

then update the critic network by minimizing loss as

$$\theta_i \leftarrow \text{argmin}_{\theta_i} B^{-1} \sum (y - Q_{\theta_i}(s, a))^2, \quad (36)$$

and periodically update the actor network every  $U$  update iteration by

$$\nabla_\phi J(\phi) = B^{-1} \sum \nabla_a Q_{\theta_1}(s, a) \Big|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s). \quad (37)$$

By adding the drift as a penalty, we set a safety threshold for the agent, while ensuring the agent seeks high-reliability rewards and maintains system stability.

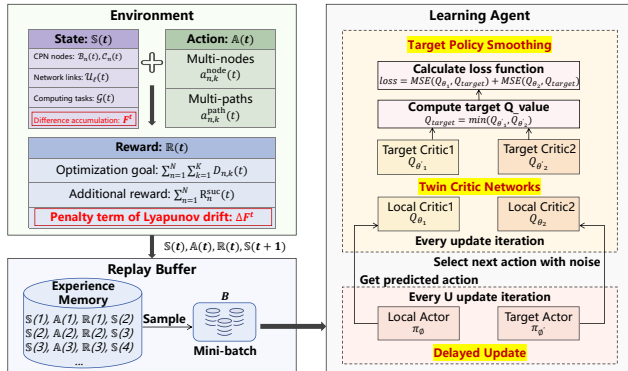


Fig. 2: The LTD3 algorithm in modified CMDP.

### IV. SIMULATION

This section evaluates the performance of the proposed LTD3 algorithm through an extensive series of simulations.

#### A. Simulation Settings

TABLE I: Parameters and values.

| Parameter in LTD3    | Value    | Parameter in the CPN                     | Value          |
|----------------------|----------|------------------------------------------|----------------|
| Training episodes    | 500      | Average number of tasks on each CPN node | 12             |
| Training steps       | 30       | Max number of tasks on each CPN node     | 15             |
| Batch size           | 16       | Upper limit of load balancing            | 0.6            |
| Actor learning rate  | 5.00E-06 | Duration of each time slot               | 2s             |
| Critic learning rate | 2.50E-05 | Maximum computing power of per CPN node  | (1, 5) GCPU/s  |
| Discount             | 9.50E-01 | Maximum cache resources of per CPN node  | (5, 20) MB     |
| Noise clip           | 0.25     | Computing volume of per computing task   | (0.01, 1) GCPU |
| Explore noise        | 0.2      | Data volume of per computing task        | (0.5, 5) MB    |
| Policy noise         | 1        | Maximum link capacity                    | (10, 100) MB   |

In our simulation, we utilize the Networkx package in Python to model a CPN with 20 CPN nodes and CPN routers. Additional parameters can be found in Table I. We implement the following algorithms for comparison.

- Lyapunov-optimized DDPG (LDDPG): optimize based on DDPG.
- No Load Balancing TD3 (NLBT): the load balancing constraints are not considered.
- Always Shortest Path TD3 (ASPT): always select the shortest SR path for routing.
- Rand Scheduling (RAND): randomly choose CPN nodes and SR paths.

#### B. Simulation Result

In Fig. 3, to demonstrate the superiority of the TD3 over the DDPG and to assess the influence of the learning rate (LR) on the algorithm's performance, we present a graphical representation of the convergence trends of cumulative rewards for both algorithms throughout the training process. Each algorithm is tested five times under different random seeds, with average values represented by solid lines and variations highlighted by shaded regions. Higher learning rates lead to quicker convergence but with pronounced oscillations, whereas lower rates, although slower, have the potential for greater rewards and stability. Using the same learning rate, LTD3 outperforms LDDPG in both Stability and comprehensive rewards, highlighting the notable improvements TD3 offers over DDPG in the context of the provided simulation settings.

Fig. 4 showcases the distribution of reward and drift values for various algorithms via a box plot, with each assessed 20 times. Ranking by overall performance, the sequence from superior to inferior is LTD3, LDDPG, ASPT, NLBT, and RAND. LDDPG trails LTD3 in performance, and its broader distribution suggests LTD3's likely stability. The decline in ASPT's performance stems from its preference for the shortest SR path, leading to CPNs overloads and subsequent cascading effects. NLBT's subpar results arise from neglecting drift optimization during training. Without optimization, RAND displays increased variability and diminished rewards.

In Fig. 5, panels (a) through (d) illustrate the evolving trends in reward, delay, success rate, and drift as the number



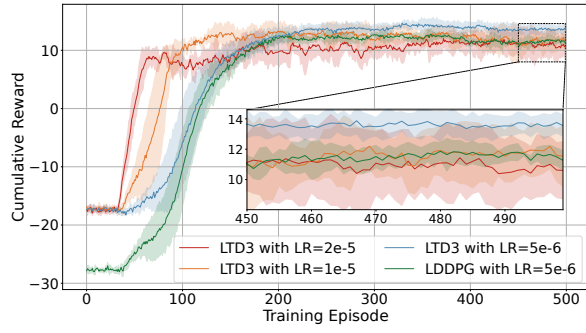


Fig. 3: Illustration of the convergence trend with different LR.

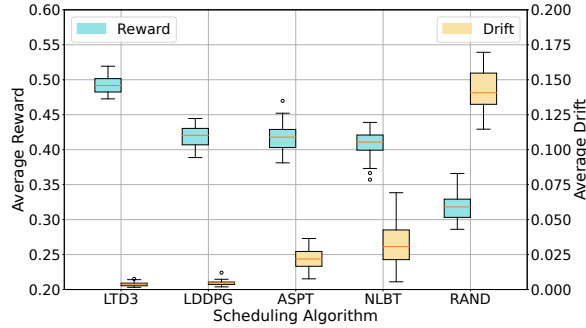


Fig. 4: Illustration of the average reward and drift with different algorithms.

of tasks rises from 8 to 16. As the number increases, the overall reward shows a decline, accompanied by a decrease in success rate and an increase in both delay and drift values. Although LTD3 exhibits a marginally lower success rate and drift performance compared to LDDPG in certain instances, it consistently achieves optimal delay and overall reward. ASPT always takes the shortest SR path, effectively reducing the transmission delay and improving the success rate when the task volume is small. However, as the task volume increases, the shortest SR path becomes congested, leading to a decrease in the transmission rate and resulting in the increased delay and a drop in success rate. NLBT overlooks the impact of load

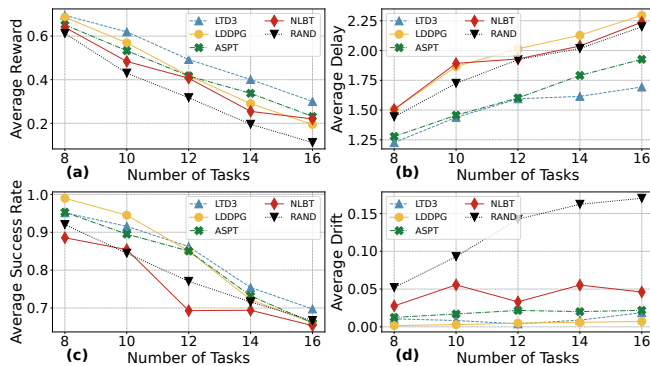


Fig. 5: Illustration of the comprehensive performance with different number of tasks under different algorithms.

balancing during the optimization of delay and success rate, leading to the subpar performance across all metrics.

## V. CONCLUSION

In this paper, we designed a delay-prioritized task scheduling policy with a dual selection of CPN nodes and SR paths that makes full use of both computing and network resources. Then, we formulated the scheduling process as a CMDP aimed at minimizing execution delay and packet loss while maintaining long-term load balancing. Leveraging the proposed Lyapunov-optimized DRL algorithm, we adeptly transformed the long-term constraint of load balancing into the instant optimization of Lyapunov drift. Finally, numerical simulations were presented to validate the superior performance of our proposed policy and algorithm.

## ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China under Grant No. 92367104. (Corresponding author: Renchao Xie.)

## REFERENCES

- [1] D. Reinsel, J. Rydning, and J. F. Gantz, "Worldwide global datasphere forecast, 2021–2025: The world keeps creating more data—now, what do we do with it all," *IDC Corporate USA*, Mar. 2021.
- [2] (2023, Aug.) The 2022-2023 Global Computing Index released. [Online]. Available: <https://www.tsinghua.edu.cn/en/info/1245/12338.htm>
- [3] M. Jiang, T. Wu, Z. Wang, Y. Gong, L. Zhang, and R. P. Liu, "A multi-intersection vehicular cooperative control based on end-edge-cloud computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 3, pp. 2459–2471, Jan. 2022.
- [4] Y. Sun *et al.*, "Computing power network: A survey," *arXiv preprint arXiv:2210.06080*, Oct. 2022.
- [5] ITU-T, "Y.2501: Computing power network - framework and architecture," Sept. 2021.
- [6] Q. Tang *et al.*, "SlaTS: A service intent-aware task scheduling framework for Computing Power Networks," *IEEE Network*, pp. 1–1, Oct. 2023.
- [7] Z. Yang *et al.*, "SkyPilot: An Intercloud Broker for Sky Computing," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, Apr. 2023, pp. 437–455.
- [8] Q. Chen, F. R. Yu, T. Huang, R. Xie, J. Liu, and Y. Liu, "Joint resource allocation for software-defined networking, caching, and computing," *IEEE/ACM Trans. Networking*, vol. 26, no. 1, pp. 274–287, Feb. 2018.
- [9] X. Ma, A. Zhou, S. Zhang, Q. Li, A. X. Liu, and S. Wang, "Dynamic task scheduling in cloud-assisted mobile edge computing," *IEEE Trans. Mob. Comput.*, vol. 22, no. 4, pp. 2116–2130, Apr. 2023.
- [10] R. Xie, D. Gu, Q. Tang, T. Huang, and F. R. Yu, "Workflow scheduling in serverless edge computing for the industrial internet of things: A learning approach," *IEEE Trans. Ind. Inf.*, vol. 19, no. 7, pp. 8242–8252, Jul. 2023.
- [11] X. Li and K. L. Yeung, "Traffic engineering in segment routing networks using milp," *IEEE Trans. Netw. Serv. Manage.*, vol. 17, no. 3, pp. 1941–1953, Sept. 2020.
- [12] Q. Tang *et al.*, "Collective deep reinforcement learning for intelligence sharing in the internet of intelligence-empowered edge computing," *IEEE Trans. Mob. Comput.*, vol. 22, no. 11, pp. 6327–6342, Nov. 2023.
- [13] R. C. B. Rego and F. M. U. d. Araújo, "Nonlinear control system with reinforcement learning and neural networks based lyapunov functions," *IEEE Lat. Am. Trans.*, vol. 19, no. 8, pp. 1253–1260, Aug. 2021.
- [14] W. Wu, P. Yang, W. Zhang, C. Zhou, and X. Shen, "Accuracy-guaranteed collaborative dnn inference in industrial iot via deep reinforcement learning," *IEEE Trans. Ind. Inf.*, vol. 17, no. 7, pp. 4988–4998, Jul. 2021.
- [15] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, Jul. 2018, pp. 1587–1596.